

Ragnarok

Architecture & Plugins

Build and solve PyPSA energy-system models in the browser.

React + TypeScript frontend · FastAPI + PyPSA + linopy backend · HiGHS solver

What is Ragnarok?

- A local, browser-based GUI for building and solving PyPSA power-system models.
- Frontend (React + TypeScript) runs on your machine; the backend solves with HiGHS.
- Models are spreadsheets: PyPSA component sheets + Ragnarok config sheets.
- Open / import an .xlsx workbook, edit, Run, analyse, and export -- full round-trip.
- Extensible through frontend-only plugins (no backend coupling).

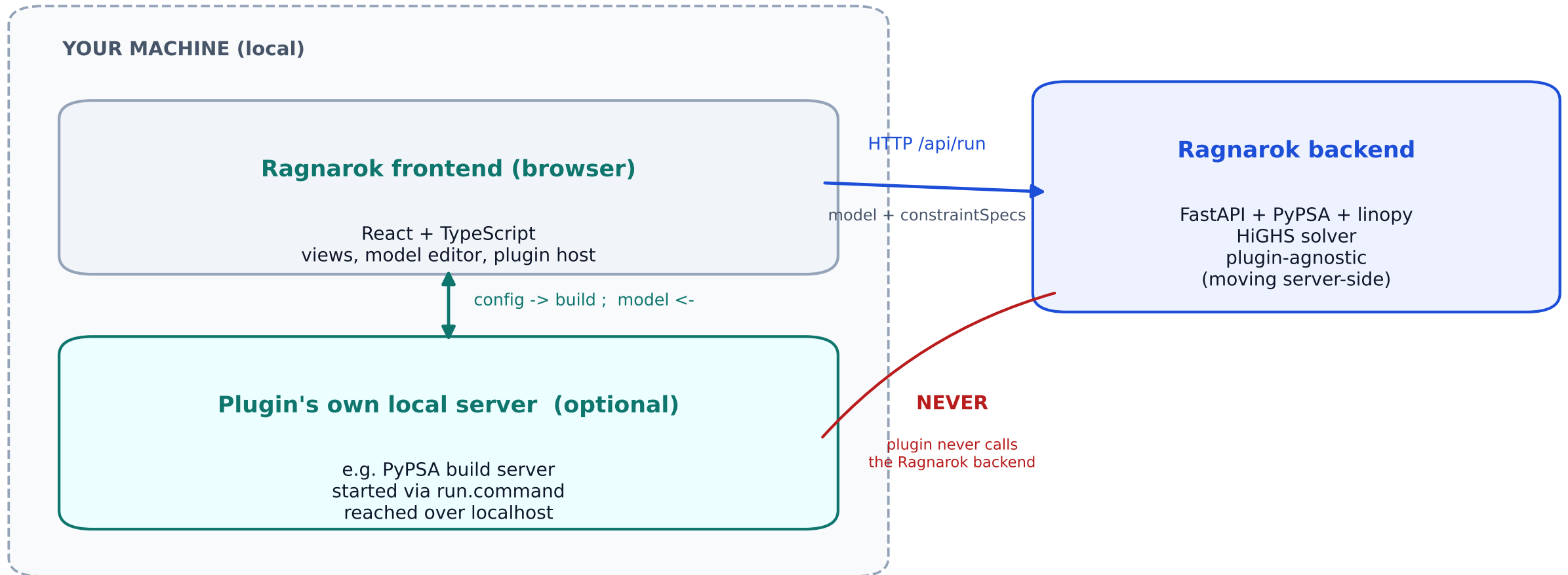
What you can do

- Build a network: buses, generators, loads, lines, links, storage, processes.
- Solve single-period dispatch, multi-year (pathway) planning, and rolling horizon.
- Apply policy: carbon price, native global constraints, and a custom-constraint DSL.
- Run stochastic and security-constrained (SCLOPF) studies.
- Explore results: KPI dashboards, time-series charts, maps, and cross-run comparison.
- Import / export the full input + output workbook entirely on the frontend.

The five views

- Build (B) -- author the model step by step, on a grid or a map.
- Model (M) -- the raw component sheet tables.
- Settings (S) -- scenarios, window, planning, rolling, carbon, constraints, solver.
 - Standard Constraints = the `global_constraints` sheet (CO2 caps, expansion limits).
 - Advanced Constraints = a custom linear-constraint DSL.
- Analytics (A) -- validation, KPIs, dashboards, and run comparison.
- Plugins (P) -- install / uninstall plugins and drive their GUIs.

Architecture & communication topology



Rule: plugin <-> Ragnarok frontend OK · frontend <-> Ragnarok backend OK · plugin <-> Ragnarok backend NEVER

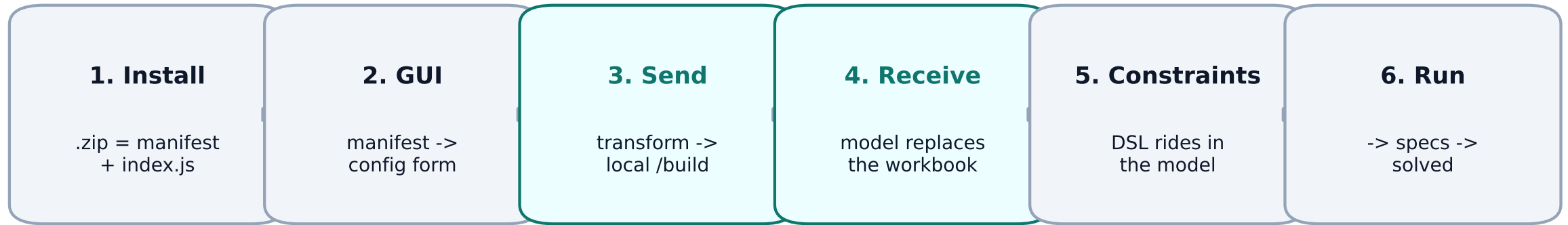
Constraints

- Standard: rows in the `global_constraints` sheet -- handled natively by PyPSA.
- Advanced: a human-friendly DSL, one linear constraint per line. Examples:
 - `gen(coal) <= 200000      cf("solar") <= 0.5      emissions <= 0.4 * gen`
- The frontend compiles the DSL to a `constraintSpecs` JSON and sends it on Run.
- Plugin-produced constraints arrive as a `RAGNAROK_CustomDSL` sheet, then flow the same way.
- Applied constraints (and shadow prices) are reported back after each solve.

Plugins: what they are

- A plugin is a .zip: a module.json manifest + a JavaScript entry (index.js).
- Installed in the browser (Plugins tab). Install / Uninstall only -- no enable toggle.
- The host renders the plugin's GUI from the manifest config schema.
- JS hooks run in the browser: transform (replace model), contribute (merge + constraints),
 - analyze (read run output), plus named action hooks (e.g. connect).
- A plugin may run its own local server for heavy work -- never the Ragnarok backend.

How a plugin flows into the model



Server registration

The plugin's heavy server is registered once in the Ragnarok project's `plugins.env` (`<absolute server dir>|<run command>`). `run.command` launches every registered server on startup, auto-using the plugin's own `.venv` if present. The plugin connects to it over localhost.

Constraints from a plugin (capacity factors, custom limits) ride inside the built model as a `RAGNAROK_CustomDSL` sheet, show up editable in Advanced Constraints, and apply on the next Run.

How to build a plugin

- 1. Write module.json: id, name, entry, a config schema (fields + tables + visibleWhen),
 - a panel layout, and -- if it has a server -- a server block (run / cwd / port / health).
- 2. Write index.js (CommonJS): export transform / contribute / analyze and any action hooks.
- 3. Optional: put heavy compute in your own backend server reached over localhost.
- 4. Zip module.json + index.js and Install it in the Plugins tab.
- 5. Register the server in plugins.env; run.command launches it. Click Connect, then Run.
- Full guide: docs/plugin.md

Example: Dashboard Importer

- A real plugin that turns a PyPSA dashboard workbook into a Ragnarok model.
- Rich manifest GUI: settings + reference tables (aggregation rules, CF limits, carbon).
- Heavy build (topology, aggregation, scaling) runs in the plugin's own FastAPI server.
- Send model -> POST config to localhost build server -> model returned to the frontend.
- CF limits (max_cf / min_cf) become `cf("carrier") <= / >=` lines in Advanced Constraints.
- The Ragnarok backend is never contacted by the plugin.

NEXT

Where to read more

- docs/user-manual.md -- install, launch, every view and feature, capabilities.
- docs/architecture.md -- tech stack, topology, data flow, process logic, design.
- docs/backend.md -- HTTP API, solve pipeline, network build, modes, constraints.
- docs/frontend.md -- App state, views, features, and the plugin host.
- docs/plugin.md -- the full plugin authoring guide.